CLA-3 – Case Study

Machine Learning – Classification

Case study title	Iris Flower Species Classification Using k-Nearest Neighbours (kNN)
Base paper	T. Cover and P. Hart, “Nearest neighbor pattern classification,” IEEE Trans. Inf. Theory, 1967
Date of submission	08-11-2026
GitHub link	[paste your GitHub repository link here]

Roll No.	Name
AP24110012103	Mani Madhav Sai

1. Introduction and Problem Statement

Assigning an object to a category from a few measurements is one of the most common tasks in machine learning, from medical diagnosis to species identification. Botanists identify iris flowers by measuring the sepals and petals, but doing this by eye is slow and subjective. A model that predicts the species directly from the measurements is a simple, clear example of a classification problem, and it shows the full workflow of data, training, testing and evaluation.

Question answered in this case study: How accurately can a k-Nearest Neighbours (kNN) classifier predict the species of an iris flower (setosa, versicolor or virginica) from its sepal and petal measurements, and which species does it confuse?

2. Base Paper Summary

Cover and Hart [1] proposed the nearest neighbour rule: a new sample is given the class of the closest stored labelled sample, and the rule extends naturally to k neighbours voting. The key idea is that no model has to be fitted; the labelled examples themselves act as the classifier, on the assumption that similar inputs tend to have similar labels.

The results reported in the paper are theoretical. The authors proved that, as the number of stored samples grows, the error rate R of the nearest neighbour rule is bounded by $R^* \le R \le 2R^*(1 - R^*)$ for two classes, where R^* is the Bayes error, the lowest error any classifier can reach. In simple terms, the nearest neighbour rule never makes more than about twice the minimum possible number of errors. The paper does not report experiments on the Iris data; the experiments in this case study are my own.

3. Method / Model Explanation

kNN is a “lazy” learner: it stores the training data and does the work only when a new flower must be classified. The steps are:

1. Store all labelled training flowers (four measurements and a species label each).
2. Standardise every measurement using the training mean and standard deviation, so that no feature dominates the distance.
3. For a new flower, compute its distance to every stored flower.
4. Select the k closest flowers (its neighbours).
5. Predict the species that appears most often among those k neighbours (majority vote).

The distance between a new flower x and a stored flower x′ is the Euclidean distance, and the prediction is the majority class among the k nearest:

$$d(x, x') = \sqrt{[\,(x_1 - x'_1)^2 + (x_2 - x'_2)^2 + (x_3 - x'_3)^2 + (x_4 - x'_4)^2\,]}$$

$$\hat{y} = \text{argmax over } c \text{ of } \Sigma\, 1(y_i = c), \text{ summed over } i \in N_k(x)$$

$$z = (x - \mu) / \sigma$$

Here x_1 to x_4 are sepal length, sepal width, petal length and petal width (cm); x' is a stored training flower; $N_k(x)$ is the set of the k training flowers closest to x; y_i is the species of the i-th neighbour; c is one candidate species; $1(\cdot)$ equals 1 when the condition is true and 0 otherwise; $\hat{y}$ is the predicted species; μ and σ are the mean and standard deviation of a feature in the training set; and z is the standardised value.

Worked example (k = 5). For simple arithmetic the example uses raw centimetres; the final model standardises first. Take the test flower x = (4.4, 3.0, 1.3, 0.2). Its three nearest training flowers are shown below. For the first one, $d = \sqrt{(0^2 + 0.1^2 + 0.1^2 + 0^2)} = \sqrt{0.02} = 0.141$.

Neighbour (x′)	Species	Distance d	Rank
(4.4, 2.9, 1.4, 0.2)	setosa	0.141	1
(4.3, 3.0, 1.1, 0.1)	setosa	0.245	2
(4.6, 3.1, 1.5, 0.2)	setosa	0.300	3

The 4th and 5th neighbours (distances 0.300 and 0.361) are also setosa, so the vote is setosa 5, versicolor 0, virginica 0, and the prediction is **setosa**, which is the true species.

4. Implementation

- **Tools:** Python with scikit-learn 1.9.1 (KNeighborsClassifier, StandardScaler, Pipeline, train_test_split) [2], NumPy 2.5.4 and Matplotlib 3.11.2. The code was written by me using scikit-learn and is available at: [paste your GitHub repository link here] (file iris_knn.py).
- **Dataset:** Fisher's Iris dataset [3], also in the UCI repository [4], loaded with sklearn.datasets.load_iris(). It has **150 examples** (50 per species), four numeric features in cm and no missing values.
- **Settings:** stratified 80/20 split with random_state = 42, giving 120 training flowers (40 per species) and 30 test flowers (10 per species). Euclidean distance, uniform voting weights, StandardScaler fitted on training data only. k was chosen from 1 to 15 using 5-fold cross-validation on the training set only.
- **Steps followed:** load data → split → build a Pipeline (StandardScaler + kNN) → cross-validate k → fit with the best k → predict on the test set → compute accuracy, confusion matrix and precision/recall/F1 → inspect correct and incorrect predictions.
- **Problems and solutions:** (i) To avoid choosing k by looking at the test set, I selected k by cross-validation on the training data only. (ii) To avoid data leakage from scaling, the scaler and classifier were put in one Pipeline so the scaler is fitted only on the training part of each fold. (iii) k = 5, 10 and 12 tied for the best CV accuracy (96.7%); I took the smallest, k = 5.

5. Results and Discussion

Cross-validation on the training set gave 94.2% for k = 1 and 96.7% for k = 5 (Figure 1). With k = 5 the model was evaluated once on the 30 unseen test flowers.

Model (k = 5)	Test accuracy	Correct / 30
kNN with standardised features (main model)	93.3%	28
kNN with raw features (comparison only)	100.0%	30

Table 1. Test accuracy of the two kNN variants

Species (main model)	Precision	Recall	F1-score
setosa	1.00	1.00	1.00
versicolor	0.83	1.00	0.91
virginica	1.00	0.80	0.89

Table 2. Per-class results on the test set (10 flowers per species)

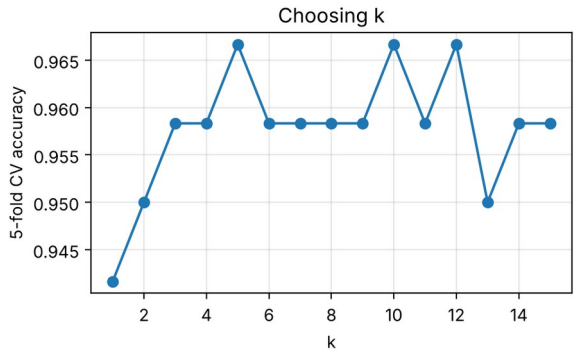


Figure 1. 5-fold CV accuracy for k = 1 to 15 (training set)

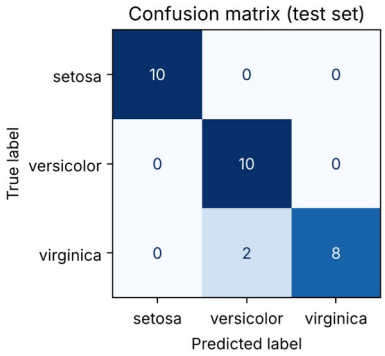


Figure 2. Confusion matrix on the 30 test flowers

Correct output: test flower (4.4, 3.0, 1.3, 0.2), true species setosa, predicted setosa. All 5 nearest neighbours are setosa, which have very small petals and are far from the other two species, so the vote is unanimous.

Incorrect output: test flower (6.0, 3.0, 4.8, 1.8), true species virginica, predicted versicolor. Its 5 neighbours were 2 virginica and 3 versicolor, so the majority vote chose the wrong species. Its petal length and width lie in the region where versicolor and virginica overlap, so even the nearest neighbour (virginica) was outvoted. A second virginica flower (6.1, 2.6, 5.6, 1.4) was also predicted as versicolor in the same way.

What the results show: the model separated setosa perfectly, and all errors were virginica flowers predicted as versicolor (Figure 2). The raw-feature model scored 100% while the standardised one scored 93.3%, but this is a difference of only 2 flowers out of 30 and is not strong evidence that scaling hurts, since all four features are already in cm on similar scales.

Limitations: the dataset is small and clean; the result comes from one random split with only 30 test flowers, so another split could give a different accuracy; k was tuned on just 120 flowers; and the raw-feature model was compared only after seeing test results, so it would be wrong to choose it because of its test score. I therefore report the standardised model, chosen beforehand, as the main result.

6. Conclusion

This case study asked how accurately kNN can predict iris species from four measurements. With k = 5 and standardised features, the model reached 96.7% cross-validated accuracy on the training data and 93.3% (28 of 30) on the unseen test flowers. Setosa was identified perfectly, while all errors were virginica flowers confused with versicolor, whose measurements overlap. The results show that kNN is a simple and effective classifier for this problem, although the small test set means the exact accuracy should be treated as approximate.

7. References

[1] T. Cover and P. Hart, “Nearest neighbor pattern classification,” IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, Jan. 1967.

[2] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] R. A. Fisher, “The use of multiple measurements in taxonomic problems,” Annals of Eugenics, vol. 7, no. 2, pp. 179–188, 1936.

[4] D. Dua and C. Graff, “Iris,” UCI Machine Learning Repository, 2019. [Online]. Available: https://archive.ics.uci.edu/dataset/53/iris

[5] Mani Madhav Sai, “Iris kNN case study code,” GitHub, 2026. [Online]. Available: [paste your GitHub repository link here]

8. Declaration

I confirm that this case study is my own work and that all results reported were obtained by me.

Signature: Mani Madhav Sai

Roll No.: AP24110012103

Name: Mani Madhav Sai